

1. What is Backend Development?

Definition:

Backend is the part of a website or application that works **behind the scenes**. Users cannot see it, but it handles all the important tasks like storing data, processing requests, and sending responses.

Example:

When you log in to Instagram or Amazon:

- You enter your username and password.
- The backend checks if the details are correct.
- If they are correct, it logs you in.

The backend does all this work.

2. Frontend vs Backend

Frontend	Backend
Visible to users	Not visible to users
HTML, CSS, JavaScript	Java, Node.js, Python, C#, PHP
Buttons, Forms, UI	Database, APIs, Business Logic

Example:

- Frontend = Restaurant Waiter 🍽️
- Backend = Chef 👨‍🍳

The waiter takes your order and gives you food.

The chef prepares the food.

3. What is Full Stack?

A Full Stack Developer works on both:

- Frontend
- Backend

Example:

A Full Stack Developer can build a complete website from start to finish.

4. Client-Server Architecture

Every web application has two parts:

Client

- User
- Browser
- Mobile App

Server

- Stores data
- Runs backend code
- Sends responses

Client (Browser)

|

Request

↓

Server (Backend)

|

Response

↓

Client

5. Request and Response

Whenever a user performs an action, a request is sent to the server.

The server processes the request and sends a response.

Example:

User clicks **Login**

↓

Browser sends request

↓

Backend checks username and password

↓

Backend sends response

↓

"Login Successful"

6. Static Website vs Dynamic Website

Static Website

- Same content for everyone

- No database
- Simple HTML pages

Example

- Portfolio Website
 - College Notice Page
-

Dynamic Website

- Content changes for each user
- Uses database
- Uses backend

Example

- Facebook
 - Amazon
 - Flipkart
 - Instagram
-

Backend Responsibilities

- Authentication
- Database Management
- Business Logic
- APIs
- Security

HTTP Methods

HTTP methods tell the server what you want to do.

1. GET

Used to read/fetch data.

Example:

GET /students

👉 Get all students.

2. POST

Used to create/add new data.

Example:

POST /students

👉 Add a new student.

3. PUT

Used to update the entire data.

Example:

PUT /students/1

👉 Replace all details of Student 1.

4. PATCH

Used to update only specific fields.

Example:

PATCH /students/1

👉 Update only the student's age or phone number.

5. DELETE

Used to delete data.

Example:

DELETE /students/1

👉 Delete Student 1.

1. What is Node.js?

Node.js is a JavaScript runtime environment that allows us to run JavaScript **outside the browser**.

Normally, JavaScript runs only in a browser.

With Node.js, we can use JavaScript to build:

- Backend
- APIs
- Web Servers
- Real-time Applications

Example:

- Instagram Backend
- Netflix Backend
- Chat Applications

2. Why Node.js?

Node.js is popular because it is:

- Fast ⚡

- Easy to Learn
- Uses JavaScript for Frontend and Backend
- Good for APIs
- Handles many users at the same time
- Large Community and npm Packages

Example:

One language (JavaScript) for both frontend and backend.

3. V8 Engine

V8 is the **JavaScript Engine** developed by Google.

It converts JavaScript code into machine code so the computer can execute it quickly.

Flow

JavaScript Code



V8 Engine



Machine Code



Output

👉 That's why Node.js is very fast.

4. Event-Driven Architecture

Node.js works on **events**.

When an event happens, Node.js executes the related function.

Example:

User Clicks Login



Login Event



Backend Checks User



Response Sent

Another example:

- Button Click
- File Upload

- API Request

All are events.

5. Single Thread

Node.js uses **one main thread** to handle requests.

It doesn't create a new thread for every user.

This makes Node.js lightweight and memory efficient.

Example

100 Users



One Node.js Thread



Handles All Requests

6. Non-blocking I/O

Node.js does **not wait** for one task to finish.

It starts another task while the first one is still running.

Example

Read File



Don't Wait



Handle Another Request



File Finished



Send Response

This makes Node.js very fast.

7. Blocking vs Non-blocking

Blocking

Task 1

↓

Wait

↓

Task 2

The program waits until Task 1 finishes.

Non-blocking

Task 1

↓

Start Task 2

↓

Task 1 Finishes Later

No waiting.

Node.js follows the **Non-blocking** approach.

8. Event Loop

The **Event Loop** is the heart of Node.js.

It checks whether a task is complete.

If a task is finished, it sends the result back.

Flow

Request

↓

Node.js

↓

Long Task



Event Loop



Task Complete



Response

The Event Loop helps Node.js handle many users efficiently.

9. Where is Node.js Used?

Node.js is commonly used for:

- REST APIs
- Backend Development
- Chat Applications
- Real-time Applications
- Social Media Apps
- Streaming Platforms
- E-commerce Websites

Examples

- Netflix
- PayPal
- LinkedIn
- Uber
- Walmart

1. Running JavaScript in Node

Normally, JavaScript runs in a **browser**.

With Node.js, we can run JavaScript in the **terminal**.

Example

Create a file:

app.js

Write:

```
console.log("Hello Node.js");
```

Run:

```
node app.js
```

Output

Hello Node.js

2. Variables

Variables are used to **store data**.

Example

```
let name = "Mukund";  
const age = 22;
```

```
console.log(name);  
console.log(age);
```

Types

- let → Value can change.
 - const → Value cannot change.
-

3. Data Types

Data Types tell what kind of data is stored.

```
let name = "Mukund"; // String  
let age = 22;        // Number  
let isStudent = true; // Boolean  
let city = null;     // Null  
let x;               // Undefined
```

Common Data Types:

- String
- Number
- Boolean
- Null
- Undefined

- Object
 - Array
-

4. Functions

A function is a **block of code** that performs a specific task.

Example

```
function greet() {  
  console.log("Welcome");  
}
```

```
greet();
```

Output

Welcome

5. Arrow Functions

Arrow functions are a **shorter way** to write functions.

Example

```
const greet = () => {  
  console.log("Hello");  
};
```

```
greet();
```

6. Template Literals

Template literals make it easy to combine text and variables.

Use **backticks (`)**.

Example

```
let name = "Mukund";
```

```
console.log(`Welcome ${name}`);
```

Output

Welcome Mukund

7. Objects

Objects store data in **key-value pairs**.

Example

```
const student = {  
  name: "Mukund",  
  age: 22,  
  course: "CSE"  
};
```

```
console.log(student.name);
```

Output

Mukund

8. Arrays

Arrays store **multiple values** in one variable.

Example

```
let fruits = ["Apple", "Mango", "Banana"];
```

```
console.log(fruits[0]);
```

Output

Apple

9. Loops

Loops repeat the same task multiple times.

Example

```
for(let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Output

1
2
3
4
5

10. Condition Statements

Conditions help us make decisions.

Example

```
let age = 20;

if(age >= 18){
  console.log("Eligible");
}
else{
  console.log("Not Eligible");
}
```

Output

Eligible

11. Modules

A module is a **separate JavaScript file** that contains reusable code.

Instead of writing all code in one file, we split it into multiple files.

Example:

math.js
app.js

12. require()

require() is used to **import a module**.

math.js

```
function add(a, b){
  return a + b;
}
```

```
module.exports = add;
```

app.js

```
const add = require("./math");
```

```
console.log(add(10, 20));
```

Output

30

13. module.exports

module.exports is used to **export** functions or variables so that other files can use them.

Example

```
function greet(){  
  console.log("Hello");  
}
```

```
module.exports = greet;
```

Now any file can use it with `require()`.